

DCC-GARCH without S&P 500

DCC-GARCH Analysis without S&P 500

This file estimates a DCC-GARCH model for five Asian stock markets only: Shanghai Composite, Hang Seng, Nikkei 225, KOSPI and Singapore STI.

For direct comparability with the extended specification, both DCC-GARCH models use the same trading-date sample obtained by inner-joining the five Asian markets and the S&P 500. The S&P 500 determines the common dates but is not included as a variable in this baseline DCC-GARCH model.

The main purpose is to examine the dynamic conditional correlations between the Chinese stock market and other Asian markets.

1. Load Packages

```
packages <- c(
  "readr", "dplyr", "lubridate", "rugarch", "rmgarch",
  "xts", "zoo", "ggplot2", "tidyr", "gt", "knitr"
)
```

```
# Load packages
lapply(packages, library, character.only = TRUE)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

date, intersect, setdiff, union

Loading required package: parallel

Attaching package: 'rmgarch'

The following objects are masked from 'package:dplyr':

first, last

Loading required package: zoo

Attaching package: 'zoo'

The following objects are masked from 'package:base':

as.Date, as.Date.numeric

Warning from 'xts' package #####
#####

#

#

The dplyr lag() function breaks how base R's lag() function is supposed to

work, which breaks lag(my_xts). Calls to lag(my_xts) that you type or

#

source() into this session won't work correctly.

#

#

#

Use stats::lag() to make sure you're not using dplyr::lag(), or you can add

conflictRules('dplyr', exclude = 'lag') to your .Rprofile to stop

#

dplyr from breaking base R's lag() function.

#

#

#

Code in packages is not affected. It's protected by R's namespace mechanism

Set `options(xts.warn\_dplyr\_breaks\_lag = FALSE)` to suppress this warning.

#

#

#####

Attaching package: 'xts'

The following objects are masked from 'package:rmgarch':

first, last

The following objects are masked from 'package:dplyr':

first, last

```
[[1]]
[1] "readr"      "stats"      "graphics"   "grDevices"  "utils"      "datasets"
[7] "methods"    "base"

[[2]]
[1] "dplyr"      "readr"      "stats"      "graphics"   "grDevices"  "utils"
[7] "datasets"  "methods"    "base"

[[3]]
[1] "lubridate" "dplyr"      "readr"      "stats"      "graphics"   "grDevices"
[7] "utils"      "datasets"  "methods"    "base"

[[4]]
[1] "rugarch"    "parallel"   "lubridate"  "dplyr"      "readr"      "stats"
[7] "graphics"   "grDevices" "utils"      "datasets"  "methods"    "base"

[[5]]
[1] "rmgarch"    "rugarch"    "parallel"   "lubridate"  "dplyr"      "readr"
[7] "stats"      "graphics"   "grDevices" "utils"      "datasets"  "methods"
[13] "base"

[[6]]
[1] "xts"        "zoo"        "rmgarch"    "rugarch"    "parallel"   "lubridate"
[7] "dplyr"      "readr"      "stats"      "graphics"   "grDevices"  "utils"
[13] "datasets"  "methods"    "base"

[[7]]
[1] "xts"        "zoo"        "rmgarch"    "rugarch"    "parallel"   "lubridate"
[7] "dplyr"      "readr"      "stats"      "graphics"   "grDevices"  "utils"
[13] "datasets"  "methods"    "base"

[[8]]
[1] "ggplot2"    "xts"        "zoo"        "rmgarch"    "rugarch"    "parallel"
```

```

[7] "lubridate" "dplyr"      "readr"      "stats"      "graphics"  "grDevices"
[13] "utils"      "datasets"   "methods"    "base"

[[9]]
[1] "tidyr"      "ggplot2"    "xts"        "zoo"        "rmgarch"    "rugarch"
[7] "parallel"   "lubridate"  "dplyr"      "readr"      "stats"      "graphics"
[13] "grDevices" "utils"      "datasets"   "methods"    "base"

[[10]]
[1] "gt"          "tidyr"      "ggplot2"    "xts"        "zoo"        "rmgarch"
[7] "rugarch"     "parallel"   "lubridate"  "dplyr"      "readr"      "stats"
[13] "graphics"    "grDevices" "utils"      "datasets"   "methods"    "base"

[[11]]
[1] "knitr"       "gt"          "tidyr"      "ggplot2"    "xts"        "zoo"
[7] "rmgarch"     "rugarch"    "parallel"   "lubridate"  "dplyr"      "readr"
[13] "stats"       "graphics"    "grDevices" "utils"      "datasets"    "methods"
[19] "base"

```

2. Import Daily Index Price Data

The repository includes a frozen local dataset ending on 2023-08-31. Rendering this document does not download or update market data.

```

data_candidates <- c(
  "../dataset/outputs/daily_prices_all_markets.csv",
  "dataset/outputs/daily_prices_all_markets.csv",
  "daily_prices_all_markets.csv"
)
data_path <- data_candidates[file.exists(data_candidates)][1]

if (is.na(data_path)) {
  stop("The frozen daily_prices_all_markets.csv included with the project was not found.")
}

daily_prices <- readr::read_csv(data_path)

Rows: 4296 Columns: 7
— Column specification —

```

```
Delimiter: ","
```

```
dbl (6): STI, Hang_Seng, KOSPI, Nikkei_225, Shanghai_Composite, SP500  
date (1): Date
```

i Use ``spec()`` to retrieve the full column specification for this data.
i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
head(daily_prices)
```

```
# A tibble: 6 × 7  
  Date          STI Hang_Seng KOSPI Nikkei_225 Shanghai_Composite SP500  
  <date>      <dbl>   <dbl> <dbl>    <dbl>         <dbl> <dbl>  
1 2003-01-06 1332.    9666.  667.    8713.         1335.  929.  
2 2003-01-07 1319.    9652.  652.    8656.         1332.  923.  
3 2003-01-08 1332.    9688.  652.    8518.         1372.  910.  
4 2003-01-09 1335.    9675.  630.    8498.         1397.  928.  
5 2003-01-10 1347.    9722.  628.    8470.         1385.  928.  
6 2003-01-14 1401.    9796.  650.    8553.         1467.  932.
```

```
str(daily_prices)
```

```
spc_tbl_ [4,296 × 7] (S3: spec_tbl_df/tbl_df/tbl/data.frame)  
$ Date          : Date[1:4296], format: "2003-01-06" "2003-01-07"  
...  
$ STI           : num [1:4296] 1332 1319 1332 1335 1347 ...  
$ Hang_Seng     : num [1:4296] 9666 9652 9688 9675 9722 ...  
$ KOSPI         : num [1:4296] 667 652 652 630 628 ...  
$ Nikkei_225    : num [1:4296] 8713 8656 8518 8498 8470 ...  
$ Shanghai_Composite: num [1:4296] 1335 1332 1372 1397 1385 ...  
$ SP500         : num [1:4296] 929 923 910 928 928 ...  
- attr(*, "spec")=  
.. cols(  
..   Date = col_date(format = ""),  
..   STI = col_double(),  
..   Hang_Seng = col_double(),  
..   KOSPI = col_double(),  
..   Nikkei_225 = col_double(),  
..   Shanghai_Composite = col_double(),  
..   SP500 = col_double()  
.. )  
- attr(*, "problems")=<externalptr>
```

3. Clean Date and Restrict Sample Period

```
daily_prices <- daily_prices %>%  
  mutate(Date = as.Date(Date)) %>%  
  arrange(Date) %>%  
  filter(  
    Date >= as.Date("2003-01-01"),
```

```

    Date <= as.Date("2023-08-31")
  )

head(daily_prices)

# A tibble: 6 × 7
  Date          STI Hang_Seng KOSPI Nikkei_225 Shanghai_Composite SP500
  <date>      <dbl>   <dbl> <dbl>   <dbl>         <dbl> <dbl>
1 2003-01-06 1332.    9666.  667.    8713.         1335.  929.
2 2003-01-07 1319.    9652.  652.    8656.         1332.  923.
3 2003-01-08 1332.    9688.  652.    8518.         1372.  910.
4 2003-01-09 1335.    9675.  630.    8498.         1397.  928.
5 2003-01-10 1347.    9722.  628.    8470.         1385.  928.
6 2003-01-14 1401.    9796.  650.    8553.         1467.  932.

tail(daily_prices)

# A tibble: 6 × 7
  Date          STI Hang_Seng KOSPI Nikkei_225 Shanghai_Composite SP500
  <date>      <dbl>   <dbl> <dbl>   <dbl>         <dbl> <dbl>
1 2023-08-24 3181.   18212. 2538.   32287.         3082. 4376.
2 2023-08-25 3190.   18119. 2519.   31624.         3064. 4406.
3 2023-08-28 3214.   18131. 2543.   32170.         3099. 4433.
4 2023-08-29 3223.   18484. 2552.   32227.         3136. 4498.
5 2023-08-30 3220.   18483. 2561.   32333.         3137. 4515.
6 2023-08-31 3233.   18382. 2556.   32619.         3120. 4508.

summary(daily_prices)

      Date          STI          Hang_Seng          KOSPI
Min.   :2003-01-06  Min.   :1214  Min.   : 8409  Min.   : 515.2
1st Qu.:2008-03-02  1st Qu.:2593  1st Qu.:18734  1st Qu.:1481.3
Median :2013-04-24  Median :3043  Median :21905  Median :1958.2
Mean   :2013-05-02  Mean   :2852  Mean   :21301  Mean   :1863.5
3rd Qu.:2018-07-05  3rd Qu.:3230  3rd Qu.:24616  3rd Qu.:2161.0
Max.   :2023-08-31  Max.   :3876  Max.   :33154  Max.   :3305.2
      Nikkei_225  Shanghai_Composite  SP500
Min.   : 7055  Min.   :1012  Min.   : 676.5
1st Qu.:10773  1st Qu.:2094  1st Qu.:1224.3
Median :15883  Median :2862  Median :1586.4
Mean   :16669  Mean   :2700  Mean   :2046.7
3rd Qu.:21318  3rd Qu.:3247  3rd Qu.:2713.0
Max.   :33753  Max.   :6092  Max.   :4793.5

colSums(is.na(daily_prices))

      Date          STI          Hang_Seng          K
OSPI
      0
      0
      0
      0

```

```

      Nikkei_225 Shanghai_Composite      SP500
      0          0          0

sum(duplicated(daily_prices$Date))

[1] 0

nrow(daily_prices)

[1] 4296

```

4. Calculate Daily Log Returns

This step converts daily price indices into daily log returns. The S&P 500 is excluded in this version.

```

daily_returns <- daily_prices %>%
  mutate(
    r_Shanghai = (log(Shanghai_Composite) - log(lag(Shanghai_Composit
e))) * 100,
    r_Hang_Seng = (log(Hang_Seng) - log(lag(Hang_Seng))) * 100,
    r_Nikkei_225 = (log(Nikkei_225) - log(lag(Nikkei_225))) * 100,
    r_KOSPI = (log(KOSPI) - log(lag(KOSPI))) * 100,
    r_STI = (log(STI) - log(lag(STI))) * 100
  ) %>%
  select(
    Date,
    r_Shanghai,
    r_Hang_Seng,
    r_Nikkei_225,
    r_KOSPI,
    r_STI
  ) %>%
  na.omit()

head(daily_returns)

# A tibble: 6 × 6
  Date      r_Shanghai r_Hang_Seng r_Nikkei_225 r_KOSPI  r_STI
  <date>      <dbl>      <dbl>      <dbl>    <dbl>  <dbl>
1 2003-01-07   -0.196    -0.140    -0.654  -2.20  -0.972
2 2003-01-08    2.96      0.370    -1.62   -0.0736 1.02
3 2003-01-09    1.83     -0.132    -0.234  -3.33   0.208
4 2003-01-10   -0.900     0.475    -0.324  -0.324  0.901
5 2003-01-14    5.75      0.767     0.971   3.39   3.94
6 2003-01-15   -0.474     0.785     0.684  -0.271 -1.06

tail(daily_returns)

# A tibble: 6 × 6
  Date      r_Shanghai r_Hang_Seng r_Nikkei_225 r_KOSPI  r_STI

```

	<date>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2023-08-24	0.125	2.03	0.861	1.28	0.206
2	2023-08-25	-0.591	-0.511	-2.07	-0.733	0.288
3	2023-08-28	1.12	0.0626	1.71	0.959	0.743
4	2023-08-29	1.19	1.93	0.177	0.343	0.292
5	2023-08-30	0.0399	-0.00633	0.330	0.354	-0.0891
6	2023-08-31	-0.552	-0.547	0.880	-0.193	0.405

```
summary(daily_returns)
```

Date	r_Shanghai	r_Hang_Seng
Min. :2003-01-07	Min. :-12.76357	Min. :-14.69537
1st Qu.:2008-03-03	1st Qu.: -0.70900	1st Qu.: -0.69851
Median :2013-04-25	Median : 0.03665	Median : 0.05591
Mean :2013-05-02	Mean : 0.01977	Mean : 0.01496
3rd Qu.:2018-07-05	3rd Qu.: 0.78061	3rd Qu.: 0.76043
Max. :2023-08-31	Max. : 9.03425	Max. : 14.24307

r_Nikkei_225	r_KOSPI	r_STI
Min. :-12.92448	Min. :-11.17200	Min. :-10.62797
1st Qu.: -0.69144	1st Qu.: -0.56747	1st Qu.: -0.48536
Median : 0.06842	Median : 0.07836	Median : 0.04508
Mean : 0.03073	Mean : 0.03129	Mean : 0.02065
3rd Qu.: 0.83989	3rd Qu.: 0.71746	3rd Qu.: 0.52959
Max. : 13.66694	Max. : 11.28435	Max. : 15.47189

```
colSums(is.na(daily_returns))
```

Date	r_Shanghai	r_Hang_Seng	r_Nikkei_225	r_KOSPI
r_STI	0	0	0	0
0				

```
nrow(daily_returns)
```

```
[1] 4295
```

4.1 Daily Data Audit and Descriptive Statistics

The audit table records how the common daily price sample becomes the return sample used in the DCC-GARCH estimation. The descriptive table is based on the five Asian return series included in the baseline model.

```
daily_data_audit <- data.frame(
  Stage = c("Common six-market daily prices", "Baseline daily returns"),
  Observations = c(nrow(daily_prices), nrow(daily_returns)),
  Start_Date = c(as.character(min(daily_prices$Date)), as.character(min(
    daily_returns$Date))),
  End_Date = c(as.character(max(daily_prices$Date)), as.character(max(d
    aily_returns$Date))),
  Missing_Values = c(sum(is.na(daily_prices)), sum(is.na(daily_return
    s))),
```



```
Duplicate_Dates = c(sum(duplicated(daily_prices$Date)), sum(duplicate
d(daily_returns$Date)))
)
```

```
daily_data_audit
```

	Stage	Observations	Start_Date	End_Date
1	Common six-market daily prices	4296	2003-01-06	2023-08-31
2	Baseline daily returns	4295	2003-01-07	2023-08-31

	Missing_Values	Duplicate_Dates
1	0	0
2	0	0

```
return_variables <- setdiff(names(daily_returns), "Date")
```

```
daily_descriptive_statistics <- do.call(rbind, lapply(return_variables,
function(variable) {
  x <- daily_returns[[variable]]
  n <- length(x)
  x_mean <- mean(x)
  x_sd <- sd(x)
  skewness <- mean((x - x_mean)^3) / x_sd^3
  excess_kurtosis <- mean((x - x_mean)^4) / x_sd^4 - 3
  jb_statistic <- n / 6 * (skewness^2 + excess_kurtosis^2 / 4)
```

```
data.frame(
  Variable = variable,
  N = n,
  Mean = x_mean,
  Std_Dev = x_sd,
  Minimum = min(x),
  Maximum = max(x),
  Skewness = skewness,
  Excess_Kurtosis = excess_kurtosis,
  JB_Statistic = jb_statistic,
  JB_p_value = pchisq(jb_statistic, df = 2, lower.tail = FALSE)
)
)))
```

```
daily_descriptive_statistics
```

	Variable	N	Mean	Std_Dev	Minimum	Maximum	Skewness
1	r_Shanghai	4295	0.01976957	1.647485	-12.76357	9.034251	-0.41447743
2	r_Hang_Seng	4295	0.01496542	1.576037	-14.69537	14.243074	0.01113167
3	r_Nikkei_225	4295	0.03073461	1.561364	-12.92448	13.666939	-0.47585199
4	r_KOSPI	4295	0.03129102	1.377895	-11.17200	11.284352	-0.53030364
5	r_STI	4295	0.02065441	1.165765	-10.62797	15.471889	0.35445471

	Excess_Kurtosis	JB_Statistic	JB_p_value
1	5.380329	5303.449	0
2	10.110441	18293.392	0

3	9.215200	15359.217	0
4	7.491423	10244.703	0
5	15.618838	43746.483	0

5. Export Cleaned Daily Returns

```
write.csv(daily_returns, "daily_returns_for_dcc_no_sp500.csv", row.names = FALSE)
```

6. Prepare Data for DCC-GARCH

```
# Keep only return variables for DCC-GARCH estimation
```

```
dcc_data_no_sp500 <- daily_returns %>%
```

```
  select(
    r_Shanghai,
    r_Hang_Seng,
    r_Nikkei_225,
    r_KOSPI,
    r_STI
  )
```

```
head(dcc_data_no_sp500)
```

```
# A tibble: 6 × 5
```

	r_Shanghai	r_Hang_Seng	r_Nikkei_225	r_KOSPI	r_STI
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	-0.196	-0.140	-0.654	-2.20	-0.972
2	2.96	0.370	-1.62	-0.0736	1.02
3	1.83	-0.132	-0.234	-3.33	0.208
4	-0.900	0.475	-0.324	-0.324	0.901
5	5.75	0.767	0.971	3.39	3.94
6	-0.474	0.785	0.684	-0.271	-1.06

```
summary(dcc_data_no_sp500)
```

r_Shanghai		r_Hang_Seng		r_Nikkei_225	
Min.	:-12.76357	Min.	:-14.69537	Min.	:-12.92448
1st Qu.:	-0.70900	1st Qu.:	-0.69851	1st Qu.:	-0.69144
Median :	0.03665	Median :	0.05591	Median :	0.06842
Mean :	0.01977	Mean :	0.01496	Mean :	0.03073
3rd Qu.:	0.78061	3rd Qu.:	0.76043	3rd Qu.:	0.83989
Max. :	9.03425	Max. :	14.24307	Max. :	13.66694
r_KOSPI		r_STI			
Min.	:-11.17200	Min.	:-10.62797		
1st Qu.:	-0.56747	1st Qu.:	-0.48536		
Median :	0.07836	Median :	0.04508		
Mean :	0.03129	Mean :	0.02065		
3rd Qu.:	0.71746	3rd Qu.:	0.52959		
Max. :	11.28435	Max. :	15.47189		

7. Specify the DCC-GARCH Model

The model uses a univariate GARCH(1,1) specification for each market and then estimates a DCC-GARCH(1,1) model for the conditional correlations.

```
# Univariate GARCH(1,1) specification for each market
uspec_no_sp500 <- ugarchspec(
  mean.model = list(armaOrder = c(0, 0), include.mean = TRUE),
  variance.model = list(model = "sGARCH", garchOrder = c(1, 1)),
  distribution.model = "norm"
)

# Apply the same GARCH specification to all return series
multispec_model_no_sp500 <- multispec(replicate(ncol(dcc_data_no_sp500),
  uspec_no_sp500))

# DCC-GARCH(1,1) specification
dcc_spec_no_sp500 <- dccspec(
  uspec = multispec_model_no_sp500,
  dccOrder = c(1, 1),
  distribution = "mvnrm"
)
```

8. Estimate the DCC-GARCH Model

```
dcc_fit_no_sp500 <- dccfit(
  dcc_spec_no_sp500,
  data = dcc_data_no_sp500
)
```

```
dcc_fit_no_sp500
```

```
*-----*
*          DCC GARCH Fit          *
*-----*
```

```
Distribution      : mvnrm
Model             : DCC(1,1)
No. Parameters    : 32
[VAR GARCH DCC UncQ] : [0+20+2+10]
No. Series        : 5
No. Obs.          : 4295
Log-Likelihood    : -30263.79
Av.Log-Likelihood : -7.05
```

```
Optimal Parameters
```

```
-----
              Estimate  Std. Error  t value Pr(>|t|)
```

[r_Shanghai].mu	0.023693	0.019356	1.2241	0.220933
[r_Shanghai].omega	0.015342	0.007276	2.1085	0.034986
[r_Shanghai].alpha1	0.063647	0.014211	4.4786	0.000008
[r_Shanghai].beta1	0.932926	0.014548	64.1291	0.000000
[r_Hang_Seng].mu	0.054207	0.019007	2.8520	0.004345
[r_Hang_Seng].omega	0.031903	0.009886	3.2271	0.001251
[r_Hang_Seng].alpha1	0.072191	0.011253	6.4153	0.000000
[r_Hang_Seng].beta1	0.913969	0.012999	70.3130	0.000000
[r_Nikkei_225].mu	0.077844	0.019189	4.0566	0.000050
[r_Nikkei_225].omega	0.068653	0.018888	3.6348	0.000278
[r_Nikkei_225].alpha1	0.112878	0.018006	6.2689	0.000000
[r_Nikkei_225].beta1	0.860088	0.021532	39.9454	0.000000
[r_KOSPI].mu	0.045109	0.016031	2.8138	0.004895
[r_KOSPI].omega	0.025772	0.008260	3.1200	0.001809
[r_KOSPI].alpha1	0.076797	0.012490	6.1484	0.000000
[r_KOSPI].beta1	0.908106	0.014890	60.9893	0.000000
[r_STI].mu	0.042122	0.012678	3.3223	0.000893
[r_STI].omega	0.017094	0.005301	3.2249	0.001260
[r_STI].alpha1	0.085611	0.015761	5.4319	0.000000
[r_STI].beta1	0.901263	0.016812	53.6088	0.000000
[Joint]dcc1	0.016643	0.002442	6.8161	0.000000
[Joint]dcc1	0.970322	0.005461	177.6677	0.000000

Information Criteria

```
-----
Akaike      14.107
Bayes       14.155
Shibata     14.107
Hannan-Quinn 14.124
```

Elapsed time : 9.075952

9. Create DCC-GARCH Parameter Table

```

dcc_param_table_no_sp500 <- as.data.frame(dcc_fit_no_sp500@mfit$matcoef)

dcc_param_table_no_sp500$Parameter <- rownames(dcc_param_table_no_sp500)

colnames(dcc_param_table_no_sp500) <- c(
  "Estimate",
  "Std_Error",
  "t_value",
  "p_value",
  "Parameter"
)

dcc_param_table_no_sp500 <- dcc_param_table_no_sp500 %>%

```

```

select(Parameter, Estimate, Std_Error, t_value, p_value) %>%
mutate(
  CI_Lower = Estimate - 1.96 * Std_Error,
  CI_Upper = Estimate + 1.96 * Std_Error
)

dcc_param_gt_no_sp500 <- dcc_param_table_no_sp500 %>%
  gt() %>%
  tab_header(
    title = "DCC-GARCH Parameter Estimates",
    subtitle = "Five Asian stock markets, excluding S&P 500"
  ) %>%
  cols_label(
    Parameter = "Parameter",
    Estimate = "Estimate",
    Std_Error = "Std. Error",
    t_value = "t value",
    p_value = "p-value",
    CI_Lower = "95% CI Lower",
    CI_Upper = "95% CI Upper"
  ) %>%
  fmt_number(
    columns = c(Estimate, Std_Error, t_value, p_value, CI_Lower, CI_Upper),
    decimals = 6
  )

dcc_param_gt_no_sp500

```

Table 1: DCC-GARCH Parameter Estimates

Five Asian stock markets, excluding S&P 500

Parameter	Estimate	Std. Error	t value	p-value	95% CI Lower	95% CI Upper
[r_Shanghai].mu	0.023693	0.019356	1.224052	0.220933	-0.014245	0.061631
[r_Shanghai].omega	0.015342	0.007276	2.108521	0.034986	0.001081	0.029604
[r_Shanghai].alpha1	0.063647	0.014211	4.478596	0.000008	0.035793	0.091502
[r_Shanghai].beta1	0.932926	0.014548	64.129064	0.000000	0.904412	0.961439
[r_Hang_Seng].mu	0.054207	0.019007	2.851992	0.004345	0.016954	0.091461
[r_Hang_Seng].omega	0.031903	0.009886	3.227077	0.001251	0.012526	0.051279
[r_Hang_Seng].alpha1	0.072191	0.011253	6.415258	0.000000	0.050135	0.094247
[r_Hang_Seng].beta1	0.913969	0.012999	70.312999	0.000000	0.888492	0.939446
[r_Nikkei_225].mu	0.077844	0.019189	4.056594	0.000050	0.040232	0.115455
[r_Nikkei_225].omega	0.068653	0.018888	3.634819	0.000278	0.031633	0.105673
[r_Nikkei_225].alpha1	0.112878	0.018006	6.268865	0.000000	0.077586	0.148170
[r_Nikkei_225].beta1	0.860088	0.021532	39.945438	0.000000	0.817886	0.902289
[r_KOSPI].mu	0.045109	0.016031	2.813839	0.004895	0.013688	0.076530
[r_KOSPI].omega	0.025772	0.008260	3.119965	0.001809	0.009582	0.041962
[r_KOSPI].alpha1	0.076797	0.012490	6.148416	0.000000	0.052315	0.101278
[r_KOSPI].beta1	0.908106	0.014890	60.989289	0.000000	0.878922	0.937289
[r_STI].mu	0.042122	0.012678	3.322341	0.000893	0.017272	0.066971
[r_STI].omega	0.017094	0.005301	3.224854	0.001260	0.006705	0.027484
[r_STI].alpha1	0.085611	0.015761	5.431911	0.000000	0.054720	0.116502
[r_STI].beta1	0.901263	0.016812	53.608828	0.000000	0.868312	0.934215
[Joint]dcc1	0.016643	0.002442	6.816135	0.000000	0.011858	0.021429
[Joint]dcc1	0.970322	0.005461	177.667681	0.000000	0.959617	0.981026

```
gtsave(
  dcc_param_gt_no_sp500,
  filename = "dcc_garch_parameter_estimates_no_sp500.png"
)
```

file:///C:/Users/20945/AppData/Local/Temp/Rtmp8qdhTS/file82c05be71dc8.html screenshot completed

10. Extract Dynamic Conditional Correlations

```
# Extract dynamic correlation array
dcc_cor_no_sp500 <- rcor(dcc_fit_no_sp500)

# Check dimensions: market x market x time
dim(dcc_cor_no_sp500)

[1] 5 5 4295
```

11. Average Dynamic Conditional Correlation Matrix

```
avg_dcc_cor_matrix_no_sp500 <- apply(dcc_cor_no_sp500, c(1, 2), mean, na.rm = TRUE)

colnames(avg_dcc_cor_matrix_no_sp500) <- colnames(dcc_data_no_sp500)
rownames(avg_dcc_cor_matrix_no_sp500) <- colnames(dcc_data_no_sp500)

knitr::kable(
  avg_dcc_cor_matrix_no_sp500,
  digits = 4,
  caption = "Average Dynamic Conditional Correlation Matrix without S&P 500"
)
```

Average Dynamic Conditional Correlation Matrix without S&P 500

	r_Shanghai	r_Hang_Seng	r_Nikkei_225	r_KOSPI	r_STI
r_Shanghai	1.0000	0.4969	0.2767	0.3057	0.3057
r_Hang_Seng	0.4969	1.0000	0.5418	0.6130	0.6607
r_Nikkei_225	0.2767	0.5418	1.0000	0.6224	0.5535
r_KOSPI	0.3057	0.6130	0.6224	1.0000	0.5674
r_STI	0.3057	0.6607	0.5535	0.5674	1.0000

12. Pairwise Dynamic Correlation Table

This table reports the average dynamic conditional correlation for each market pair.

```
make_corr_row <- function(x, pair_name) {
  x <- na.omit(x)
  coef <- mean(x)
  se <- sd(x) / sqrt(length(x))
  z_value <- coef / se
  p_value <- 2 * (1 - pnorm(abs(z_value)))
  ci_lower <- coef - 1.96 * se
  ci_upper <- coef + 1.96 * se

  data.frame(
    Pair = pair_name,
    Coef = coef,
    Std_Error = se,
    z = z_value,
    P_value = p_value,
    CI_Lower = ci_lower,
    CI_Upper = ci_upper
  )
}
```

```

market_names <- colnames(dcc_data_no_sp500)
pair_index <- combn(seq_along(market_names), 2)

pairwise_corr_table <- do.call(
  rbind,
  lapply(seq_len(ncol(pair_index)), function(i) {
    m1 <- pair_index[1, i]
    m2 <- pair_index[2, i]
    pair_name <- paste0("corr(", market_names[m1], ", ", market_names[m
2], ")")
    make_corr_row(dcc_cor_no_sp500[m1, m2, ], pair_name)
  })
)

pairwise_corr_gt <- pairwise_corr_table %>%
  gt() %>%
  tab_header(
    title = "Pairwise Dynamic Conditional Correlations",
    subtitle = "DCC-GARCH estimated correlations without S&P 500"
  ) %>%
  cols_label(
    Pair = "Pair",
    Coef = "Coef.",
    Std_Error = "Std. Err.",
    z = "z",
    P_value = "P>|z|",
    CI_Lower = "95% CI Lower",
    CI_Upper = "95% CI Upper"
  ) %>%
  fmt_number(
    columns = c(Coef, Std_Error, z, P_value, CI_Lower, CI_Upper),
    decimals = 4
  ) %>%
  tab_options(
    table.font.size = 13,
    heading.title.font.size = 16,
    heading.subtitle.font.size = 13
  )

pairwise_corr_gt

```


Table 2: Pairwise Dynamic Conditional Correlations

DCC-GARCH estimated correlations without S&P 500

Pair	Coef.	Std. Err.	z	P> z	95% CI Lower	95% CI Upper
corr(r_Shanghai, r_Hang_Seng)	0.4969	0.0017	291.4741	0.0000	0.4936	0.5003
corr(r_Shanghai, r_Nikkei_225)	0.2767	0.0014	193.0042	0.0000	0.2739	0.2795
corr(r_Shanghai, r_KOSPI)	0.3057	0.0016	192.0133	0.0000	0.3026	0.3088
corr(r_Shanghai, r_STI)	0.3057	0.0017	181.8933	0.0000	0.3024	0.3090
corr(r_Hang_Seng, r_Nikkei_225)	0.5418	0.0014	400.2296	0.0000	0.5392	0.5445
corr(r_Hang_Seng, r_KOSPI)	0.6130	0.0012	528.8249	0.0000	0.6108	0.6153
corr(r_Hang_Seng, r_STI)	0.6607	0.0015	451.3469	0.0000	0.6578	0.6636
corr(r_Nikkei_225, r_KOSPI)	0.6224	0.0013	473.5418	0.0000	0.6198	0.6249
corr(r_Nikkei_225, r_STI)	0.5535	0.0012	465.1556	0.0000	0.5512	0.5558
corr(r_KOSPI, r_STI)	0.5674	0.0013	443.9980	0.0000	0.5649	0.5699

```
gtsave(
  pairwise_corr_gt,
  filename = "pairwise_dcc_correlations_no_sp500.png"
)
```

file:///C:/Users/20945/AppData/Local/Temp/Rtmp8qdhTS/file82c06260acd.html screenshot completed

13. Create Daily Dynamic Correlation Dataset

This section focuses on the correlations between Shanghai and the other Asian markets.

```
dcc_daily <- data.frame(
  Date = daily_returns$Date,
  corr_SH_Hang_Seng = dcc_cor_no_sp500[1, 2, ],
  corr_SH_Nikkei = dcc_cor_no_sp500[1, 3, ],
  corr_SH_KOSPI = dcc_cor_no_sp500[1, 4, ],
  corr_SH_STI = dcc_cor_no_sp500[1, 5, ]
)
```

```
head(dcc_daily)
```

	Date	corr_SH_Hang_Seng	corr_SH_Nikkei	corr_SH_KOSPI
1970-01-02	2003-01-07	0.5082895	0.2914966	0.3196472
1970-01-03	2003-01-08	0.5083161	0.2918112	0.3161112
1970-01-04	2003-01-09	0.5002763	0.2459889	0.3047523
1970-01-05	2003-01-10	0.4934346	0.2405322	0.2486602
1970-01-06	2003-01-14	0.4892333	0.2428446	0.2506471
1970-01-07	2003-01-15	0.4750236	0.2567399	0.3262980
	corr_SH_STI			
1970-01-02		0.3221872		
1970-01-03		0.3219000		
1970-01-04		0.3367272		

```
1970-01-05    0.3358335
1970-01-06    0.3258420
1970-01-07    0.4428253
```

```
summary(dcc_daily)
```

	Date	corr_SH_Hang_Seng	corr_SH_Nikkei	corr_SH_KOSPI
Min.	:2003-01-07	Min. :0.1170	Min. :0.01428	Min. :0.006
1st Qu.	:2008-03-03	1st Qu.:0.4426	1st Qu.:0.21344	1st Qu.:0.230
Median	:2013-04-25	Median :0.5280	Median :0.27286	Median :0.315
Mean	:2013-05-02	Mean :0.4969	Mean :0.27673	Mean :0.305
3rd Qu.	:2018-07-05	3rd Qu.:0.5754	3rd Qu.:0.33884	3rd Qu.:0.377
Max.	:2023-08-31	Max. :0.6959	Max. :0.54729	Max. :0.615
corr_SH_STI				
Min.		Min. :-0.01575		
1st Qu.		1st Qu.: 0.23328		
Median		Median : 0.30969		
Mean		Mean : 0.30573		
3rd Qu.		3rd Qu.: 0.38357		
Max.		Max. : 0.62951		

14. Average Shanghai-Market Dynamic Correlations

```
avg_shanghai_cor <- data.frame(
  Market_Pair = c(
    "Shanghai-Hang Seng",
    "Shanghai-Nikkei",
    "Shanghai-KOSPI",
    "Shanghai-STI"
  ),
  Average_DCC_Correlation = c(
    mean(dcc_daily$corr_SH_Hang_Seng, na.rm = TRUE),
    mean(dcc_daily$corr_SH_Nikkei, na.rm = TRUE),
    mean(dcc_daily$corr_SH_KOSPI, na.rm = TRUE),
    mean(dcc_daily$corr_SH_STI, na.rm = TRUE)
  )
)
```

```
avg_shanghai_cor
```

	Market_Pair	Average_DCC_Correlation
1	Shanghai-Hang Seng	0.4969260
2	Shanghai-Nikkei	0.2767263

3	Shanghai-KOSPI	0.3056938
4	Shanghai-STI	0.3057281

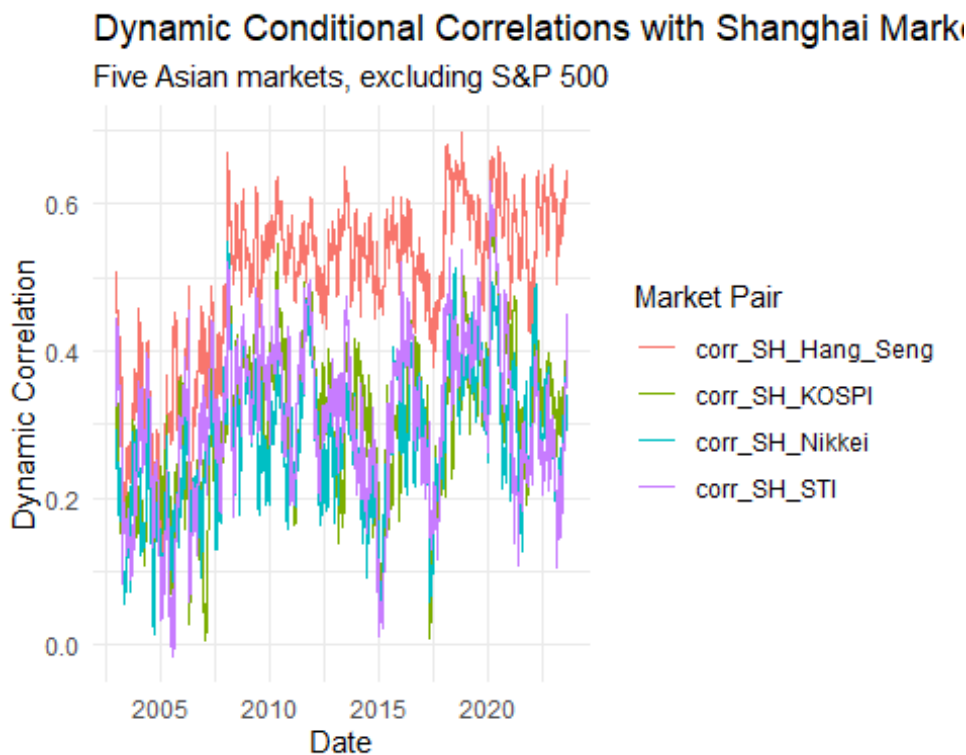
15. Export Daily Dynamic Correlations

```
write.csv(dcc_daily, "dcc_dynamic_correlations_daily_no_sp500.csv", row.
names = FALSE)
```

16. Plot All Shanghai Dynamic Correlations

```
dcc_long <- dcc_daily %>%
  pivot_longer(
    cols = starts_with("corr_"),
    names_to = "Market_Pair",
    values_to = "Correlation"
  )

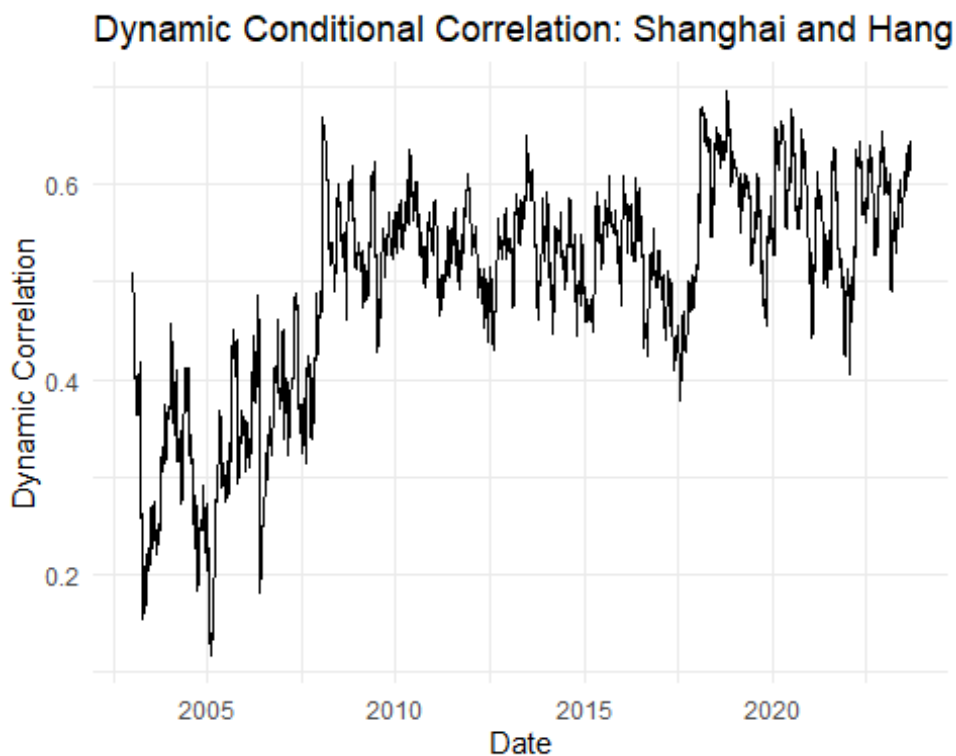
ggplot(dcc_long, aes(x = Date, y = Correlation, color = Market_Pair)) +
  geom_line(linewidth = 0.5) +
  labs(
    title = "Dynamic Conditional Correlations with Shanghai Market",
    subtitle = "Five Asian markets, excluding S&P 500",
    x = "Date",
    y = "Dynamic Correlation",
    color = "Market Pair"
  ) +
  theme_minimal()
```



17. Plot Individual Shanghai Dynamic Correlations

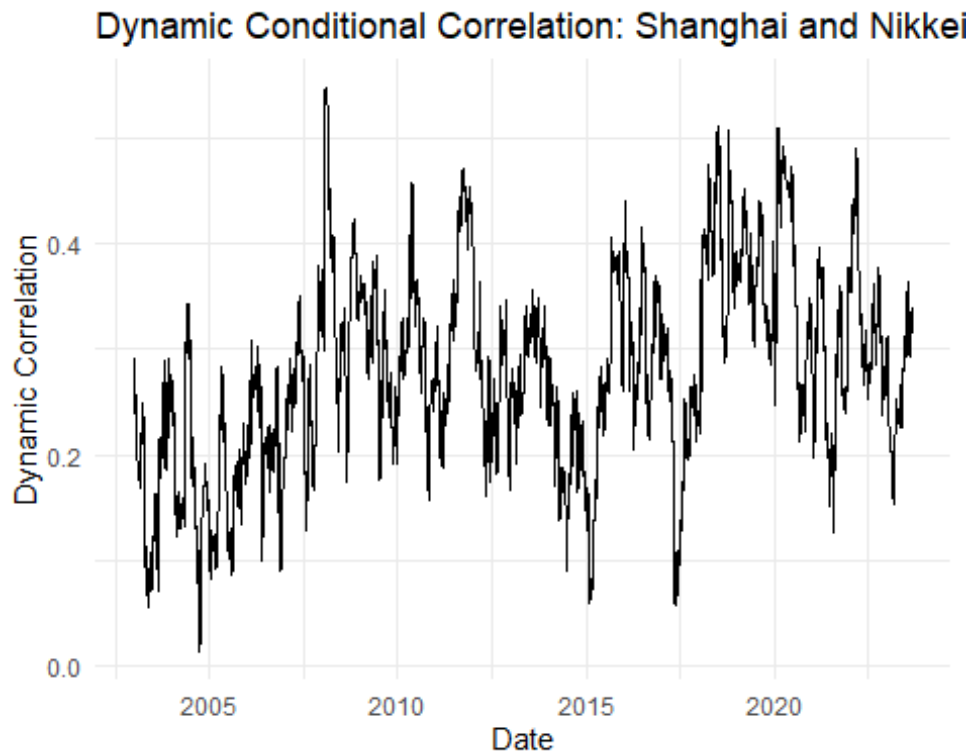
```
# Shanghai-Hang Seng
plot_sh_hs <- ggplot(dcc_daily, aes(x = Date, y = corr_SH_Hang_Seng)) +
  geom_line(linewidth = 0.5) +
  labs(
    title = "Dynamic Conditional Correlation: Shanghai and Hang Seng",
    x = "Date",
    y = "Dynamic Correlation"
  ) +
  theme_minimal()
```

plot_sh_hs



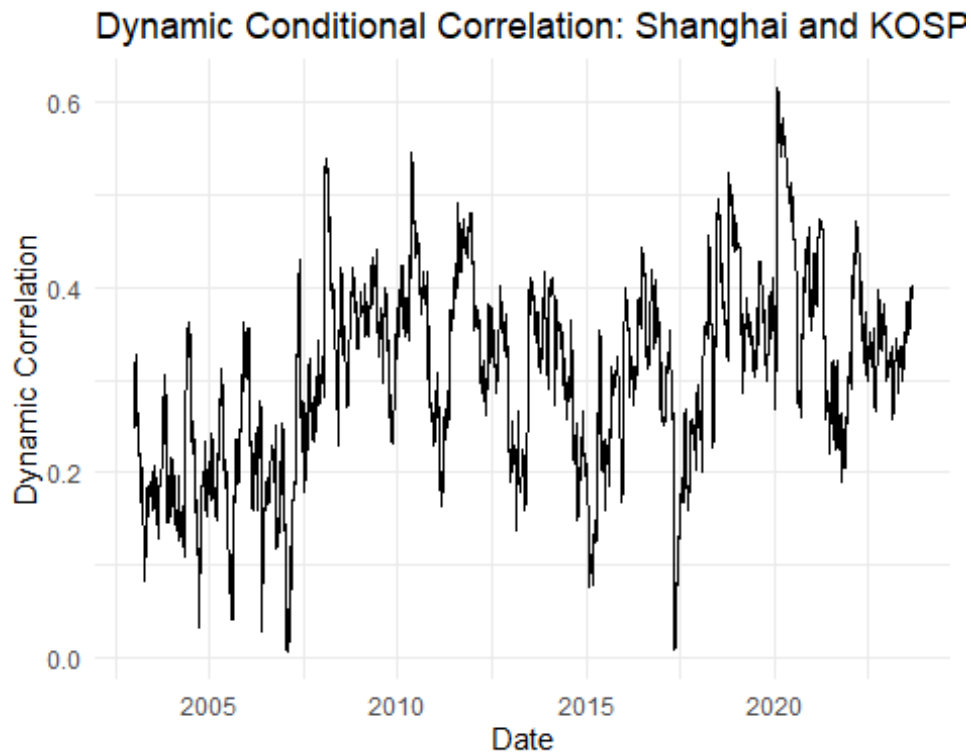
```
# Shanghai-Nikkei
plot_sh_nikkei <- ggplot(dcc_daily, aes(x = Date, y = corr_SH_Nikkei)) +
  geom_line(linewidth = 0.5) +
  labs(
    title = "Dynamic Conditional Correlation: Shanghai and Nikkei 225",
    x = "Date",
    y = "Dynamic Correlation"
  ) +
  theme_minimal()
```

plot_sh_nikkei



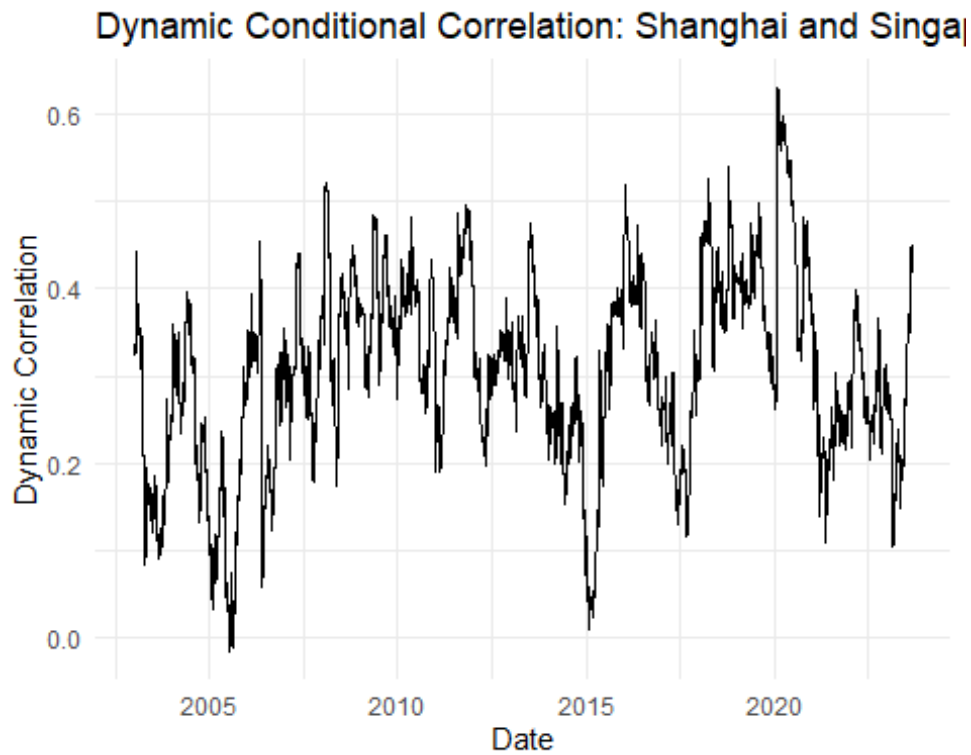
```
# Shanghai-KOSPI
plot_sh_kospi <- ggplot(dcc_daily, aes(x = Date, y = corr_SH_KOSPI)) +
  geom_line(linewidth = 0.5) +
  labs(
    title = "Dynamic Conditional Correlation: Shanghai and KOSPI",
    x = "Date",
    y = "Dynamic Correlation"
  ) +
  theme_minimal()

plot_sh_kospi
```



```
# Shanghai-STI
plot_sh_sti <- ggplot(dcc_daily, aes(x = Date, y = corr_SH_STI)) +
  geom_line(linewidth = 0.5) +
  labs(
    title = "Dynamic Conditional Correlation: Shanghai and Singapore ST
I",
    x = "Date",
    y = "Dynamic Correlation"
  ) +
  theme_minimal()

plot_sh_sti
```



18. Save Individual Plots

```
ggsave("dcc_shanghai_hang_seng_no_sp500.png", plot_sh_hs, width = 8, height = 5, dpi = 300)
ggsave("dcc_shanghai_nikkei_no_sp500.png", plot_sh_nikkei, width = 8, height = 5, dpi = 300)
ggsave("dcc_shanghai_kospi_no_sp500.png", plot_sh_kospi, width = 8, height = 5, dpi = 300)
ggsave("dcc_shanghai_sti_no_sp500.png", plot_sh_sti, width = 8, height = 5, dpi = 300)
```